JUMHC Portal - Beginner Edition

A simpler, from-scratch MERN stack walkthrough

Companion learning document

---

0. How This Document Works

This is a **simplified** version of a full production-style walkthrough. It builds the exact same idea — a mountaineering club website with a public site and an admin dashboard — but with easier code, fewer edge cases, and fewer moving parts, so you can actually type it out yourself and understand every line.

What got simplified compared to a "real" production app, and why:

Removed / Simplified	Why
No file uploads (Multer)	You just paste an image link (like an Unsplash URL) instead of uploading a photo. Uploading files adds a whole new topic (multipart forms, disk storage) that isn't needed to learn CRUD + auth.
No email sending (Nodemailer)	Sending real emails needs SMTP accounts and credentials. We just `console.log` a "confirmation email" instead.
No CSV export	Nice bonus feature, not essential to learn MERN basics.
No atomic race-condition-proof seat counting	We do a simple "check then save". There's a tiny chance two people grab the last seat at the exact same millisecond — a fun problem to solve *after* you're comfortable with the basics.
One simple admin role	No "Super Admin vs Editor" permission levels.
Simple generic error messages	Still handled with try/catch everywhere, just without extra polish.

Everything else — the two-app split (backend + frontend), MongoDB + Mongoose, Express routes, JWT login, React Router, Context API, Tailwind styling — is all still here, because those are the real MERN fundamentals.

The project still has two folders that talk to each other over HTTP:

- `/backend` — a Node.js + Express server that talks to MongoDB and returns JSON.
- `/frontend` — a React app (built with Vite) that calls that API and shows pages.

We build the backend first, then the frontend, because the frontend has nothing to show until there's an API to call.

---

1. The Big Picture

MongoDB — the database, stores everything as JSON-like documents. **E**xpress — a small web framework for Node that handles HTTP requests. **R**eact — builds the actual web pages the user clicks around. **N**ode.js — the JavaScript runtime that runs Express on your machine/server.

Two extra tools:

- **Vite** — runs your React app in dev mode and rebuilds it for production. Much faster than the older Create React App.
- **Tailwind CSS** — instead of writing your own CSS file, you style things directly in your JSX with classes like `bg-blue-600 px-4 py-2 rounded-lg`.

How one page load actually works, in one sentence: your React component calls `fetch()`, Express matches the URL to a route function, that function asks Mongoose to get data from MongoDB, and the JSON travels back to React's `useState`, which re-renders the page with real data.

We'll trace this exact path again at the end, once everything exists.

---

2. Project Setup: The Root Folder

Before writing backend or frontend code, we add one small file at the very top, outside both folders.

`package.json` (project root)

This isn't an app — it's just a shortcut so you can install and run both halves with one command instead of juggling two terminals.

```
{
  "name": "jumhc-portal",
  "version": "1.0.0",
  "scripts": {
    "install-all": "npm install && npm install --prefix backend && npm install --prefix
frontend",
    "backend": "npm run dev --prefix backend",
    "frontend": "npm run dev --prefix frontend",
    "dev": "concurrently \"npm run backend\" \"npm run frontend\"",
    "seed": "npm run seed --prefix backend"
  },
  "devDependencies": {
    "concurrently": "^8.2.2"
  }
}
```

- `install-all` installs dependencies in all three places in one go.
- `dev` starts the backend and frontend at the same time, using `concurrently` — a tiny tool whose only job is running two npm scripts side-by-side in one terminal window.
- `seed` fills the (empty) database with some starter data so the site isn't blank the first time you open it.

---

3. Backend, Step 1: Dependencies & Secrets

`backend/package.json`

```
{
  "name": "jumhc-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "seed": "node scripts/seed.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
```

```
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.4.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.3"
  }
}
```

Only six dependencies — every one earns its place:

- **express** — receives requests, sends responses.
- **mongoose** — lets us write `Event.find()` instead of raw MongoDB queries.
- **bcryptjs** — hashes admin passwords so they're never stored as plain text.
- **jsonwebtoken** — creates and checks the login token (JWT) that proves "yes, this is a logged-in admin".
- **cors** — without this, the browser blocks your React app (port 5173) from calling your API (port 5000), since they're different "origins".
- **dotenv** — loads secret settings from a `.env` file instead of hard-coding them.
- **nodemon** (dev only) — restarts the server automatically every time you save a file.

`backend/.env` (never committed to Git — add it to `.gitignore`)

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/jumhc
JWT_SECRET=any_long_random_string_here
```

That's it — three values. `PORT` is which port Express listens on, `MONGO_URI` is your database's address, and `JWT_SECRET` is the secret key used to sign login tokens (never share this).

---

4. Backend, Step 2: Connecting to the Database

`backend/config/db.js`

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected!');
  } catch (error) {
    console.error('Could not connect to MongoDB:', error.message);
    process.exit(1); // stop the server, there's no point running without a database
  }
};

module.exports = connectDB;
```

Simple: try to connect, log success, and if it fails, print the error and shut the whole server down — because every single route in this app needs the database, so there's no point limping along without it.

---

5. Backend, Step 3: The Server Entry Point

`backend/server.js`

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');

dotenv.config();
connectDB();

const app = express();

// Middleware
app.use(cors());
app.use(express.json()); // lets us read JSON sent from the frontend as req.body

// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/secretaries', require('./routes/secretaries'));
app.use('/api/alumni', require('./routes/alumni'));
app.use('/api/events', require('./routes/events'));
app.use('/api/registrations', require('./routes/registrations'));
app.use('/api/gallery', require('./routes/gallery'));

app.get('/', (req, res) => {
  res.send('JUMHC API is running!');
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Reading top to bottom, in the order things happen:

1. `dotenv.config()` loads your `.env` values into `process.env`.
2. `connectDB()` starts connecting to MongoDB.
3. `express.json()` is what lets `req.body` actually contain the JSON the frontend sends — forget this line and every POST/PUT route silently breaks.
4. Each `app.use('/api/...', require('./routes/...'))` line mounts one route file under a URL prefix.
5. `app.listen(...)` finally starts the server.

Beginner tip: middleware order matters. `express.json()` must run *before* any route that reads `req.body`.

---

6. Backend, Step 4: The Data Models

Each model file tells Mongoose the "shape" of one kind of document: what fields it has, and what type each one is.

backend/models/AdminUser.js

```
const mongoose = require('mongoose');

const AdminUserSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  passwordHash: { type: String, required: true }
});

module.exports = mongoose.model('AdminUser', AdminUserSchema);
```

Notice the field is called `passwordHash`, never `password` — we only ever store the bcrypt-hashed version, never the real password.

backend/models/Secretary.js

```
const mongoose = require('mongoose');

const SecretarySchema = new mongoose.Schema({
  yearStart: { type: Number, required: true },
  yearEnd: { type: Number, required: true },
  name: { type: String, required: true },
  note: { type: String }
});

module.exports = mongoose.model('Secretary', SecretarySchema);
```

backend/models/Alumni.js

```
const mongoose = require('mongoose');

const AlumniSchema = new mongoose.Schema({
  name: { type: String, required: true },
  batchYear: { type: Number, required: true },
  roleHistory: { type: String, required: true },
  bio: { type: String },
  photoUrl: { type: String }, // just a pasted image link, no file upload
  isActiveInstructor: { type: Boolean, default: false }
});

module.exports = mongoose.model('Alumni', AlumniSchema);
```

backend/models/Event.js

```
const mongoose = require('mongoose');

const EventSchema = new mongoose.Schema({
  title: { type: String, required: true },
  type: { type: String, required: true, enum: ['expedition', 'competition', 'course', 'other']
},
  date: { type: Date, required: true },
  description: { type: String, required: true },
  location: { type: String, required: true },
  registrationOpen: { type: Boolean, default: true },
  capacity: { type: Number, default: null }, // null = unlimited seats
  registrationsCount: { type: Number, default: 0 }
});

module.exports = mongoose.model('Event', EventSchema);
```

`registrationsCount` is a running counter our server updates every time someone registers — that's how we show "12 / 40 seats filled" without counting Registration documents every time the page loads.

backend/models/GalleryItem.js

```
const mongoose = require('mongoose');

const GalleryItemSchema = new mongoose.Schema({
  imageUrl: { type: String, required: true },
  year: { type: Number, required: true },
  eventTag: { type: String, required: true },
  caption: { type: String }
});

module.exports = mongoose.model('GalleryItem', GalleryItemSchema);
```

backend/models/Registration.js

```
const mongoose = require('mongoose');

const RegistrationSchema = new mongoose.Schema({
  eventId: { type: mongoose.Schema.Types.ObjectId, ref: 'Event', required: true },
  name: { type: String, required: true },
  email: { type: String, required: true },
  phone: { type: String, required: true },
  experienceLevel: { type: String, required: true },
  paymentStatus: { type: String, enum: ['Pending', 'Received'], default: 'Pending' },
  registeredAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Registration', RegistrationSchema);
```

The important bit is `eventId: { type: mongoose.Schema.Types.ObjectId, ref: 'Event' }` — instead of copying the whole event into every registration, we just store a reference (its `_id`) and look the event up when we need it.

---

7. Backend, Step 5: Middleware (Auth Guard)

"Middleware" just means a function that runs *between* a request arriving and the actual route logic — usually to check something first.

backend/middleware/auth.js

```
const jwt = require('jsonwebtoken');

const protect = (req, res, next) => {
  const authHeader = req.headers.authorization; // "Bearer <token>"

  if (!authHeader || !authHeader.startsWith('Bearer')) {
    return res.status(401).json({ message: 'Not authorized, no token' });
  }

  try {
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded; // attach the logged-in admin's info to the request
    next(); // let the request continue to the real route
  } catch (error) {
    res.status(401).json({ message: 'Not authorized, token invalid' });
  }
};

module.exports = protect;
```

`protect` looks for `Authorization: Bearer <token>`, checks the token is real and unexpired, and if it is, calls `next()` so the request continues into the actual route handler. If not, it responds with `401 Unauthorized` and stops there — `next()` never gets called, so the protected code never runs.

We'll use this on every admin-only route by simply adding it as a second argument: `router.post('/', protect, async (req, res) => {...})`.

Why a JWT instead of storing sessions in the database? A JWT carries the admin's info right inside it, signed so it can't be faked. The server can check it's valid just by re-running the same signing math — no database lookup needed.

---

8. Backend, Step 6: The API Routes

Every route file follows the same shape: make an Express `Router`, define handlers for each URL + method, export it. We'll look at `events.js` in full, since every other file repeats the same pattern (that's the beauty of CRUD — Create, Read, Update, Delete).

backend/routes/events.js

```
const express = require('express');
const router = express.Router();
const Event = require('../models/Event');
const protect = require('../middleware/auth');

// GET all events — public
router.get('/', async (req, res) => {
  try {
    const events = await Event.find().sort({ date: 1 });
    res.json(events);
  } catch (error) {
    res.status(500).json({ message: 'Server error fetching events' });
  }
});

// GET one event by id — public
router.get('/:id', async (req, res) => {
  try {
    const event = await Event.findById(req.params.id);
    if (!event) return res.status(404).json({ message: 'Event not found' });
    res.json(event);
  } catch (error) {
    res.status(500).json({ message: 'Server error fetching event' });
  }
});

// CREATE an event — admin only
router.post('/', protect, async (req, res) => {
  try {
    const newEvent = new Event(req.body);
    const saved = await newEvent.save();
    res.status(201).json(saved);
  } catch (error) {
    res.status(400).json({ message: 'Could not create event. Check required fields.' });
  }
});

// UPDATE an event — admin only
router.put('/:id', protect, async (req, res) => {
  try {
    const updated = await Event.findByIdAndUpdate(req.params.id, req.body, { new: true });
    if (!updated) return res.status(404).json({ message: 'Event not found' });
    res.json(updated);
  } catch (error) {
    res.status(400).json({ message: 'Could not update event' });
  }
});

// DELETE an event — admin only
router.delete('/:id', protect, async (req, res) => {
  try {
    const deleted = await Event.findByIdAndDelete(req.params.id);
    if (!deleted) return res.status(404).json({ message: 'Event not found' });
    res.json({ message: 'Event deleted' });
  } catch (error) {
    res.status(500).json({ message: 'Server error deleting event' });
  }
});

module.exports = router;
```

Five routes = the CRUD pattern you'll see repeated everywhere:

- `GET /` and `GET /:id` are left **public** (no `protect`) — anyone can view events.
- `POST /`, `PUT /:id`, `DELETE /:id` all have `protect` slipped in as a second argument before the handler — Express runs middleware in the order you list it, so `protect` runs first, and only calls the real handler via `next()` if the token checks out.

Beginner shortcut worth knowing: `Event.findByIdAndUpdate(id, req.body, { new: true })` is a one-line way to update just the fields the client sent, and `{ new: true }` tells Mongoose to hand back the *updated* document instead of the old one.

`backend/routes/secretaries.js` — identical shape, different model

```
const express = require('express');
const router = express.Router();
const Secretary = require('../models/Secretary');
const protect = require('../middleware/auth');

router.get('/', async (req, res) => {
  const secretaries = await Secretary.find().sort({ yearStart: -1 });
  res.json(secretaries);
});

router.post('/', protect, async (req, res) => {
  try {
    const saved = await new Secretary(req.body).save();
    res.status(201).json(saved);
  } catch (error) {
    res.status(400).json({ message: 'Could not save entry' });
  }
});

router.put('/:id', protect, async (req, res) => {
  const updated = await Secretary.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(updated);
});

router.delete('/:id', protect, async (req, res) => {
  await Secretary.findByIdAndDelete(req.params.id);
  res.json({ message: 'Deleted' });
});

module.exports = router;
```

Once you understand `events.js`, this file has nothing new to teach — same shape, `Secretary` instead of `Event`.

`backend/routes/alumni.js` and `backend/routes/gallery.js`

Since we dropped file uploads, these two are now just plain CRUD too — `photoUrl` / `imageUrl` are ordinary text fields the admin pastes a link into, exactly like every other field:

```
// backend/routes/alumni.js
const express = require('express');
const router = express.Router();
const Alumni = require('../models/Alumni');
const protect = require('../middleware/auth');

router.get('/', async (req, res) => {
  const alumni = await Alumni.find().sort({ batchYear: -1 });
  res.json(alumni);
});

router.post('/', protect, async (req, res) => {
  try {
    const saved = await new Alumni(req.body).save();
    res.status(201).json(saved);
  } catch (error) {
    res.status(400).json({ message: 'Could not save alumni profile' });
```

```
  }
});

router.put('/:id', protect, async (req, res) => {
  const updated = await Alumni.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(updated);
});

router.delete('/:id', protect, async (req, res) => {
  await Alumni.findByIdAndDelete(req.params.id);
  res.json({ message: 'Deleted' });
});

module.exports = router;
```

`gallery.js` is the same shape again, just swapping in `GalleryItem`.

`backend/routes/registrations.js` — the one with real logic

```
const express = require('express');
const router = express.Router();
const Registration = require('../models/Registration');
const Event = require('../models/Event');
const protect = require('../middleware/auth');

// PUBLIC: register for an event
router.post('/', async (req, res) => {
  const { eventId, name, email, phone, experienceLevel } = req.body;

  if (!eventId || !name || !email || !phone || !experienceLevel) {
    return res.status(400).json({ message: 'Please fill in all required fields' });
  }

  try {
    const event = await Event.findById(eventId);
    if (!event) return res.status(404).json({ message: 'Event not found' });

    if (!event.registrationOpen) {
      return res.status(400).json({ message: 'Registrations are closed for this event' });
    }

    if (event.capacity !== null && event.registrationsCount >= event.capacity) {
      return res.status(400).json({ message: 'Sorry, this course is already full!' });
    }

    const registration = await new Registration({ eventId, name, email, phone, experienceLevel
}).save();

    // Bump the seat counter
    event.registrationsCount += 1;
    await event.save();

    // In a real app you'd email a confirmation here — for now we just log it
    console.log(`New registration for "${event.title}" from ${name} (${email})`);

    res.status(201).json({ message: 'Registration successful!', registration });
  } catch (error) {
    res.status(500).json({ message: 'Server error during registration' });
  }
});

// ADMIN: view registrations for one event
router.get('/event/:eventId', protect, async (req, res) => {
  const registrations = await Registration.find({ eventId: req.params.eventId }).sort({
registeredAt: -1 });
  res.json(registrations);
});

// ADMIN: mark a registration's payment as received / pending
router.put('/:id/payment', protect, async (req, res) => {
```

```
  const updated = await Registration.findByIdAndUpdate(
    req.params.id,
    { paymentStatus: req.body.paymentStatus },
    { new: true }
  );
  res.json(updated);
});

// ADMIN: delete a registration
router.delete('/:id', protect, async (req, res) => {
  const reg = await Registration.findByIdAndDelete(req.params.id);
  if (reg) {
    // free up a seat again
    await Event.findByIdAndUpdate(reg.eventId, { $inc: { registrationsCount: -1 } });
  }
  res.json({ message: 'Registration deleted' });
});

module.exports = router;
```

Walking through the public “register” route, step by step:

1. Check every required field was actually sent.
2. Look up the event — if it doesn’t exist, stop.
3. Check registrations are still open, and there’s still room (`registrationsCount < capacity`).
4. Save the registration, then increment the event’s seat counter.
5. Log a pretend confirmation email to the console.

A note for when you’re ready to level up: step 3 and step 4 happen as two separate steps here. If two people submit the form for the very last seat within the same instant, there’s a small chance both get let in (a classic “race condition”). Real production apps close this gap with an atomic database update. It’s a genuinely fun bug to fix once you’re comfortable with the basics — for a learning project, this simpler version is completely fine.

backend/routes/auth.js

```
const express = require('express');
const router = express.Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const AdminUser = require('../models/AdminUser');
const protect = require('../middleware/auth');

router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res.status(400).json({ message: 'Please enter email and password' });
  }

  try {
    const admin = await AdminUser.findOne({ email });
    if (!admin) return res.status(401).json({ message: 'Invalid credentials' });

    const isMatch = await bcrypt.compare(password, admin.passwordHash);
    if (!isMatch) return res.status(401).json({ message: 'Invalid credentials' });

    const token = jwt.sign(
      { id: admin._id, email: admin.email },
      process.env.JWT_SECRET,
      { expiresIn: '30d' }
    );

    res.json({ token, admin: { id: admin._id, email: admin.email } });
  } catch (error) {
    res.status(500).json({ message: 'Server error' });
  }
});

// Check if a saved token is still valid, and who it belongs to
```

```
router.get('/me', protect, async (req, res) => {
  const admin = await AdminUser.findById(req.user.id).select('-passwordHash');
  res.json(admin);
});

module.exports = router;
```

Notice both “wrong email” and “wrong password” return the exact same `'Invalid credentials'` message — that’s on purpose, so nobody can use the error message to guess which email addresses have accounts.

`.select('-passwordHash')` on the `/me` route means the hashed password never gets sent back over the network, even to the admin’s own logged-in browser.

---

9. Backend, Step 7: Seeding Starter Data

backend/scripts/seed.js

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');
const dotenv = require('dotenv');
const AdminUser = require('../models/AdminUser');
const Secretary = require('../models/Secretary');
const Alumni = require('../models/Alumni');
const Event = require('../models/Event');
const GalleryItem = require('../models/GalleryItem');

dotenv.config();

const seed = async () => {
  await mongoose.connect(process.env.MONGO_URI);
  console.log('Connected. Clearing old data...');

  await AdminUser.deleteMany();
  await Secretary.deleteMany();
  await Alumni.deleteMany();
  await Event.deleteMany();
  await GalleryItem.deleteMany();

  const passwordHash = await bcrypt.hash('admin1966', 10);
  await AdminUser.create({ email: 'admin@jumhc.org', passwordHash });
  console.log('Admin created: admin@jumhc.org / admin1966');

  await Secretary.insertMany([
    { yearStart: 1966, yearEnd: 1967, name: 'Sujal K. Sen', note: 'Founding Secretary' },
    { yearStart: 2024, yearEnd: 2025, name: 'Rohan Banerjee', note: 'Post-pandemic expedition
revival' },
    { yearStart: 2025, yearEnd: 2026, name: 'Ananya Chakraborty', note: 'Current serving
Secretary' }
  ]);

  await Alumni.insertMany([
    {
      name: 'Devasis Mukhopadhyay',
      batchYear: 1999,
      roleHistory: 'Secretary 1998-2000 -> Lead Instructor',
      bio: 'Pioneered artificial wall climbing on campus.',
      photoUrl: 'https://images.unsplash.com/photo-1500648767791-00dcc994a43e?
auto=format&fit=crop&q=80&w=200',
      isActiveInstructor: true
    },
    {
      name: 'Prof. Kalyan Roy',
      batchYear: 1972,
      roleHistory: 'Secretary 1972-1974 -> Life Member',
      bio: 'Founding generation member, led treks in the Garhwal Himalayas.',
```

```
        photoUrl: 'https://images.unsplash.com/photo-1507003211169-0a1dd7228f2d?
auto=format&fit=crop&q=80&w=200',
        isActiveInstructor: false
      }
    ]);

    await Event.insertMany([
      {
        title: 'Annual Rock Climbing Course',
        type: 'course',
        date: new Date('2027-02-15'),
        description: 'A 4-day certified rock climbing course at Susunia Hills.',
        location: 'Susunia Hills, Purulia',
        registrationOpen: true,
        capacity: 40
      },
      {
        title: 'Inter-University Bouldering Championship',
        type: 'competition',
        date: new Date('2026-11-20'),
        description: 'Bouldering competition on our campus artificial wall.',
        location: 'JU Sports Complex',
        registrationOpen: true,
        capacity: 100
      }
    ]);

    await GalleryItem.insertMany([
      {
        imageUrl: 'https://images.unsplash.com/photo-1522163182402-834f871fd851?
auto=format&fit=crop&q=80&w=600',
        year: 2025,
        eventTag: 'Rock Climbing',
        caption: 'Bouldering practice at Susunia Hills.'
      }
    ]);

    console.log('Seeding done!');
    mongoose.connection.close();
  };

  seed();
```

`deleteMany()` at the top means running the seed script twice never leaves you with duplicate data. The one admin account is created with `bcrypt.hash('admin1966', 10)` so it can log in immediately with that password.

At this point — even with zero frontend code written — you already have a working JSON API. Try `npm run dev` in the backend, then `npm run seed`, then open `http://localhost:5000/api/events` in your browser to see real data.

---

10. Frontend, Step 1: Dependencies & Build Tools

frontend/package.json

```
{
  "name": "frontend",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
```

```
    "dependencies": {
      "lucide-react": "^0.379.0",
      "react": "^19.2.7",
      "react-dom": "^19.2.7",
      "react-router-dom": "^6.23.1"
    },
    "devDependencies": {
      "@vitejs/plugin-react": "^6.0.2",
      "autoprefixer": "^10.4.19",
      "postcss": "^8.4.38",
      "tailwindcss": "^3.4.3",
      "vite": "^8.1.0"
    }
  }
```

- **react-router-dom** — gives us “pages” like `/about` without a real page reload.
- **lucide-react** — a library of ready-made icon components (calendar, mail, compass icons etc.).
- **tailwindcss / postcss / autoprefixer** — the CSS pipeline.

`frontend/vite.config.js`

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
})
```

`frontend/tailwind.config.js`

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ["./index.html", "./src/**/*.{js,jsx}"],
  theme: {
    extend: {
      colors: {
        primary: { 600: '#415776', 900: '#0B2545' },
        accent: { 500: '#dc763c', 600: '#cf5e2c' }
      }
    }
  },
  plugins: []
}
```

We only define two small custom color palettes here (`primary` for navy tones, `accent` for the warm orange highlight color) instead of a full three-palette 50-950 system — enough to make the site look intentional without a huge config file to explain.

---

11. Frontend, Step 2: The HTML Shell & Entry Point

`frontend/index.html`

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>JUMHC — Jadavpur University Mountaineering & Hiking Club</title>
</head>
<body>
```

```
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

React takes over everything inside `<div id="root">`. The one `<script>` tag pulls in your whole React app.

frontend/src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

frontend/src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  @apply bg-gray-50 text-gray-800;
}
```

That's really all you need for Tailwind to start generating CSS for every class you use anywhere in `src/`.

---

12. Frontend, Step 3: The API Helper

Every page needs to talk to the backend. Instead of repeating `fetch()` boilerplate everywhere, we write it once.

frontend/src/utils/api.js

```
const BASE_URL = 'http://localhost:5000/api';

const getHeaders = () => {
  const token = localStorage.getItem('jumhc_token');
  const headers = { 'Content-Type': 'application/json' };
  if (token) headers['Authorization'] = `Bearer ${token}`;
  return headers;
};

const handleResponse = async (response) => {
  if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    throw new Error(errorData.message || 'Something went wrong');
  }
  return response.json();
};

export const api = {
  get: (endpoint) =>
    fetch(`${BASE_URL}${endpoint}`, { headers: getHeaders() }).then(handleResponse),
```

```
  post: (endpoint, data) =>
    fetch(`${BASE_URL}${endpoint}`, {
      method: 'POST',
      headers: getHeaders(),
      body: JSON.stringify(data)
    }).then(handleResponse),

  put: (endpoint, data) =>
    fetch(`${BASE_URL}${endpoint}`, {
      method: 'PUT',
      headers: getHeaders(),
      body: JSON.stringify(data)
    }).then(handleResponse),

  delete: (endpoint) =>
    fetch(`${BASE_URL}${endpoint}`, {
      method: 'DELETE',
      headers: getHeaders()
    }).then(handleResponse)
};
```

Because we skipped file uploads, this helper is much shorter than a “real” version — every request is plain JSON, so there’s no need for a special `isFormData` flag anywhere.

Every page can now just do:

```
const events = await api.get('/events');
```

...and get back real data or a thrown `Error` with a readable message, without repeating any of this plumbing.

---

13. Frontend, Step 4: Global Login State (AuthContext)

The Navbar, the admin dashboard, and the “is someone logged in” check on the login page all need to share the same piece of state. React’s Context API is built exactly for this.

frontend/src/context/AuthContext.jsx

```
import React, { createContext, useState, useEffect, useContext } from 'react';
import { api } from '../utils/api';

const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [admin, setAdmin] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const checkLogin = async () => {
      const token = localStorage.getItem('jumhc_token');
      if (!token) {
        setLoading(false);
        return;
      }
      try {
        const data = await api.get('/auth/me');
        setAdmin(data);
      } catch {
        localStorage.removeItem('jumhc_token');
      } finally {
        setLoading(false);
      }
```

```
    };
    checkLogin();
  }, []);

  const login = async (email, password) => {
    const data = await api.post('/auth/login', { email, password });
    localStorage.setItem('jumhc_token', data.token);
    setAdmin(data.admin);
  };

  const logout = () => {
    localStorage.removeItem('jumhc_token');
    setAdmin(null);
  };

  return (
    <AuthContext.Provider value={{ admin, loading, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);
```

`AuthProvider` wraps the whole app and owns the real state: `admin` (or `null`) and `loading` (true only while checking a saved token). Any component anywhere can call `useAuth()` to read `{ admin, loading, login, logout }`.

Why save the token in `localStorage`? State that lives only in React is wiped the instant the page refreshes. Saving it in `localStorage` means an admin who refreshes the dashboard doesn't get logged out — the `useEffect` above re-checks it on every fresh page load.

---

14. Frontend, Step 5: Shared Components

`frontend/src/components/Navbar.jsx`

```
import React from 'react';
import { Link, useLocation } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import { Compass, ShieldAlert, LogOut } from 'lucide-react';

const links = [
  { name: 'Home', path: '/' },
  { name: 'About', path: '/about' },
  { name: 'Alumni', path: '/alumni' },
  { name: 'Gallery', path: '/gallery' },
  { name: 'Events', path: '/events' },
  { name: 'Contact', path: '/contact' }
];

const Navbar = () => {
  const { admin, logout } = useAuth();
  const location = useLocation();

  return (
    <nav className="bg-primary-900 text-white sticky top-0 z-50 shadow">
      <div className="max-w-6xl mx-auto px-4 flex items-center justify-between h-16">
        <Link to="/" className="flex items-center gap-2 font-bold text-lg">
          <Compass className="w-6 h-6 text-accent-500" />
          JUMHC
        </Link>

        <div className="hidden md:flex gap-4 items-center">
          {links.map((link) => (
```

```
                  <Link
                    key={link.path}
                    to={link.path}
                    className={`text-sm ${location.pathname === link.path ? 'text-accent-500' : 'text-
gray-200 hover:text-white'}`}
                  >
                    {link.name}
                  </Link>
                ))}

                {admin ? (
                  <>
                    <Link to="/admin" className="flex items-center gap-1 text-sm font-bold text-green-
300">
                      <ShieldAlert className="w-4 h-4" /> Admin
                    </Link>
                    <button onClick={logout} title="Logout">
                      <LogOut className="w-5 h-5" />
                    </button>
                  </>
                ) : null}
              </div>
            </div>
          </nav>
        );
      };

      export default Navbar;
```

We skipped the mobile hamburger-menu toggle here to keep the component shorter — on small screens the links just wrap onto a second line. That's a genuinely good "next thing to add yourself" once the rest works.

frontend/src/components/ProtectedRoute.jsx

```
      import React from 'react';
      import { Navigate } from 'react-router-dom';
      import { useAuth } from '../context/AuthContext';

      const ProtectedRoute = ({ children }) => {
        const { admin, loading } = useAuth();

        if (loading) return <p className="text-center py-20">Checking login...</p>;
        if (!admin) return <Navigate to="/admin/login" replace />;

        return children;
      };

      export default ProtectedRoute;
```

This mirrors what `middleware/auth.js` does on the backend, just for the frontend: while we're still checking, show a loading message; if there's no admin, redirect to the login page; otherwise, show whatever page was wrapped inside it.

frontend/src/components/Footer.jsx

```
      import React from 'react';

      const Footer = () => (
        <footer className="bg-primary-900 text-gray-300 text-center py-6 text-sm mt-12">
          <p>&copy; {new Date().getFullYear()} JU Mountaineering & Hiking Club — Est. 1966</p>
        </footer>
      );

      export default Footer;
```

A simple one-line footer is completely fine for a learning project — you can always add contact details and

social links back in once the rest of the app works.

---

15. Frontend, Step 6: The Router (App.jsx)

`frontend/src/App.jsx`

```
import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from './context/AuthContext';
import Navbar from './components/Navbar';
import Footer from './components/Footer';
import ProtectedRoute from './components/ProtectedRoute';

import Home from './pages/Home';
import About from './pages/About';
import Alumni from './pages/Alumni';
import Gallery from './pages/Gallery';
import Events from './pages/Events';
import CourseRegistration from './pages/CourseRegistration';
import Contact from './pages/Contact';
import AdminLogin from './pages/AdminLogin';
import AdminDashboard from './pages/AdminDashboard';

function App() {
  return (
    <AuthProvider>
      <Router>
        <Navbar />
        <main className="min-h-[80vh]">
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/about" element={<About />} />
            <Route path="/alumni" element={<Alumni />} />
            <Route path="/gallery" element={<Gallery />} />
            <Route path="/events" element={<Events />} />
            <Route path="/register/:eventId" element={<CourseRegistration />} />
            <Route path="/contact" element={<Contact />} />

            <Route path="/admin/login" element={<AdminLogin />} />
            <Route
              path="/admin"
              element={
                <ProtectedRoute>
                  <AdminDashboard />
                </ProtectedRoute>
              }
            />

            <Route path="*" element={<Navigate to="/" replace />} />
          </Routes>
        </main>
        <Footer />
      </Router>
    </AuthProvider>
  );
}

export default App;
```

`AuthProvider` wraps everything so any component can call `useAuth()`. `/register/:eventId` uses a URL parameter — whatever's in that spot of the URL is readable inside `CourseRegistration.jsx` via `useParams()`, so one page can handle registration for every event. The catch-all `path="*"` route redirects any unknown URL back home.

---

16. Frontend, Step 7: The Public Pages

Almost every page below follows the same recognizable shape: `useState` for the data + a loading flag, a `useEffect` that fetches once when the page loads, and JSX that shows a loading message while waiting and the real content once data arrives. Once that shape clicks, the rest is just “what makes this page different”.

frontend/src/pages/Home.jsx

```
import React from 'react';
import { Link } from 'react-router-dom';

const Home = () => {
  return (
    <div>
      <section className="bg-primary-900 text-white py-24 text-center px-4">
        <h1 className="text-4xl sm:text-5xl font-bold mb-4">Mountaineering & Hiking Club</h1>
        <p className="text-gray-300 max-w-xl mx-auto mb-8">
          Fostering the spirit of adventure at Jadavpur University since 1966.
        </p>
        <Link to="/events" className="bg-accent-500 hover:bg-accent-600 px-6 py-3 rounded-lg
font-bold">
          View Upcoming Events
        </Link>
      </section>

      <section className="max-w-4xl mx-auto py-16 px-4 text-center">
        <h2 className="text-2xl font-bold mb-4">Welcome to JUMHC</h2>
        <p className="text-gray-600 leading-relaxed">
          Since 1966, we've trained students in rock climbing, bouldering, and led expeditions
          to some of the highest peaks in India. Explore our alumni network, browse our photo
          archive, or register for the next course.
        </p>
      </section>
    </div>
  );
};

export default Home;
```

We dropped the “find a featured event to link to” logic from the fancier version — a plain link to `/events` is simpler and works just as well; the visitor picks the exact event they want from there.

frontend/src/pages/About.jsx

```
import React, { useState, useEffect } from 'react';
import { api } from '../utils/api';

const About = () => {
  const [secretaries, setSecretaries] = useState([]);

  useEffect(() => {
    api.get('/secretaries').then(setSecretaries).catch(console.error);
  }, []);

  return (
    <div className="max-w-4xl mx-auto py-16 px-4">
      <h1 className="text-3xl font-bold mb-6">Our Heritage</h1>
      <p className="text-gray-600 leading-relaxed mb-10">
        Founded in 1966, JUMHC has sent expeditions to peaks over 6,000 meters and run our
        signature rock climbing course at Susunia Hills for nearly six decades.
      </p>

      <h2 className="text-2xl font-bold mb-4">Secretaries Timeline</h2>
      <div className="space-y-3">
```

```
              {secretaries.map((sec) => (
                <div key={sec._id} className="bg-white p-4 rounded-lg border">
                  <p className="font-bold">{sec.yearStart} – {sec.yearEnd}: {sec.name}</p>
                  {sec.note && <p className="text-sm text-gray-500">{sec.note}</p>}
                </div>
              ))}
            </div>
          </div>
        );
      };

      export default About;
```

The full version had a separate `Timeline.jsx` component with search and sort built in. For a first version, fetching the list and mapping straight over it is simpler and totally fine — you can always pull it into its own component later once you understand *why* that split is useful (reusability, keeping files smaller).

frontend/src/pages/Alumni.jsx

```
      import React, { useState, useEffect } from 'react';
      import { api } from '../utils/api';

      const Alumni = () => {
        const [alumni, setAlumni] = useState([]);
        const [loading, setLoading] = useState(true);
        const [search, setSearch] = useState('');

        useEffect(() => {
          api.get('/alumni')
            .then(setAlumni)
            .catch((err) => console.error(err))
            .finally(() => setLoading(false));
        }, []);

        const filtered = alumni.filter((a) =>
          a.name.toLowerCase().includes(search.toLowerCase())
        );

        if (loading) return <p className="text-center py-20">Loading alumni...</p>;

        return (
          <div className="max-w-5xl mx-auto py-16 px-4">
            <h1 className="text-3xl font-bold mb-6">Alumni Network</h1>

            <input
              type="text"
              placeholder="Search by name..."
              value={search}
              onChange={(e) => setSearch(e.target.value)}
              className="border rounded-lg p-2 mb-8 w-full sm:w-72"
            />

            <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-6">
              {filtered.map((member) => (
                <div key={member._id} className="bg-white p-5 rounded-xl border shadow-sm">
                  <img
                    src={member.photoUrl || 'https://images.unsplash.com/photo-1535713875002-
d1d0cf377fde?auto=format&fit=crop&q=80&w=200'}
                    alt={member.name}
                    className="w-16 h-16 rounded-full object-cover mb-3"
                  />
                  <h3 className="font-bold">{member.name}</h3>
                  <p className="text-xs text-accent-600 font-semibold">Batch {member.batchYear}</p>
                  <p className="text-sm text-gray-600 mt-2">{member.roleHistory}</p>
                  {member.isActiveInstructor && (
                    <span className="inline-block mt-2 text-xs bg-green-100 text-green-700 px-2 py-0.5
rounded-full">
                      Active Instructor
                    </span>
```

```
              ))}
            </div>
          ))}
        </div>
      </div>
    );
  };

  export default Alumni;
```

One filter (search by name) instead of two — dropping the “active instructors only” toggle keeps the state simpler while still teaching the core idea: fetch once, filter in the browser as the user types, no extra network requests needed.

frontend/src/pages/Gallery.jsx

```
  import React, { useState, useEffect } from 'react';
  import { api } from '../utils/api';

  const Gallery = () => {
    const [items, setItems] = useState([]);
    const [tag, setTag] = useState('All');

    useEffect(() => {
      api.get('/gallery').then(setItems).catch(console.error);
    }, []);

    const tags = ['All', ...new Set(items.map((i) => i.eventTag))];
    const filtered = tag === 'All' ? items : items.filter((i) => i.eventTag === tag);

    return (
      <div className="max-w-5xl mx-auto py-16 px-4">
        <h1 className="text-3xl font-bold mb-6">Gallery & Archive</h1>

        <select
          value={tag}
          onChange={(e) => setTag(e.target.value)}
          className="border rounded-lg p-2 mb-8 bg-white"
        >
          {tags.map((t) => <option key={t} value={t}>{t}</option>)}
        </select>

        <div className="grid grid-cols-2 sm:grid-cols-3 gap-4">
          {filtered.map((item) => (
            <div key={item._id} className="rounded-lg overflow-hidden border bg-white">
              <img src={item.imageUrl} alt={item.caption} className="w-full h-40 object-cover" />
              {item.caption && <p className="text-xs p-2 text-gray-600">{item.caption}</p>}
            </div>
          ))}
        </div>
      </div>
    );
  };

  export default Gallery;
```

We kept just one filter (by tag) and dropped the year filter and the full-screen lightbox popup — both are nice, but neither teaches a new concept beyond “filter an array with `.filter()`” and “toggle some state on click”, which you already get from the tag dropdown.

frontend/src/pages/Events.jsx

```
  import React, { useState, useEffect } from 'react';
  import { Link } from 'react-router-dom';
  import { api } from '../utils/api';

  const Events = () => {
```

```
          const [events, setEvents] = useState([]);
          const [loading, setLoading] = useState(true);

          useEffect(() => {
            api.get('/events')
              .then(setEvents)
              .catch(console.error)
              .finally(() => setLoading(false));
          }, []);

          const getStatus = (event) => {
            if (!event.registrationOpen) return 'Closed';
            if (event.capacity !== null && event.registrationsCount >= event.capacity) return 'Full';
            return 'Open';
          };

          if (loading) return <p className="text-center py-20">Loading events...</p>;

          return (
            <div className="max-w-4xl mx-auto py-16 px-4">
              <h1 className="text-3xl font-bold mb-8">Upcoming Events & Courses</h1>

              <div className="space-y-4">
                {events.map((event) => {
                  const status = getStatus(event);
                  return (
                    <div key={event._id} className="bg-white p-6 rounded-xl border flex flex-col
sm:flex-row justify-between gap-4">
                      <div>
                        <span className="text-xs uppercase font-bold text-gray-400">{event.type}</span>
                        <h3 className="font-bold text-lg">{event.title}</h3>
                        <p className="text-sm text-gray-600">{event.description}</p>
                        <p className="text-xs text-gray-500 mt-2">
                          {new Date(event.date).toLocaleDateString()} • {event.location}
                        </p>
                        {event.capacity !== null && (
                          <p className="text-xs text-gray-500">
                            {event.registrationsCount} / {event.capacity} seats filled
                          </p>
                        )}
                      </div>

                      <div className="flex items-center">
                        {status === 'Open' ? (
                          <Link
                            to={`/register/${event._id}`}
                            className="bg-accent-500 hover:bg-accent-600 text-white px-5 py-2.5 rounded-
lg font-bold text-sm whitespace-nowrap"
                          >
                            Register
                          </Link>
                        ) : (
                          <span className="text-gray-400 font-bold text-sm">{status}</span>
                        )}
                      </div>
                    </div>
                  );
                })}
              </div>
            </div>
          );
        };

        export default Events;
```

`getStatus()` turns a few raw fields into one simple label — “Open”, “Full”, or “Closed” — so the button and the text always agree with each other. We dropped the separate “upcoming vs past” split and the progress bar; a simple seat-count line teaches the same idea with less code.

frontend/src/pages/CourseRegistration.jsx

```
        import React, { useState, useEffect } from 'react';
        import { useParams, Link } from 'react-router-dom';
        import { api } from '../utils/api';

        const CourseRegistration = () => {
          const { eventId } = useParams();
          const [event, setEvent] = useState(null);
          const [formData, setFormData] = useState({ name: '', email: '', phone: '', experienceLevel: ''
});
          const [submitting, setSubmitting] = useState(false);
          const [success, setSuccess] = useState(false);
          const [error, setError] = useState(null);

          useEffect(() => {
            api.get(`/events/${eventId}`).then(setEvent).catch(() => setError('Could not load this
event'));
          }, [eventId]);

          const handleChange = (e) => {
            setFormData({ ...formData, [e.target.name]: e.target.value });
          };

          const handleSubmit = async (e) => {
            e.preventDefault();
            setSubmitting(true);
            setError(null);
            try {
              await api.post('/registrations', { eventId, ...formData });
              setSuccess(true);
            } catch (err) {
              setError(err.message);
            } finally {
              setSubmitting(false);
            }
          };

          if (!event) return <p className="text-center py-20">Loading...</p>;

          if (success) {
            return (
              <div className="max-w-md mx-auto py-20 text-center px-4">
                <h2 className="text-2xl font-bold mb-3">Registration Successful!</h2>
                <p className="text-gray-600 mb-6">Thanks for signing up for {event.title}.</p>
                <Link to="/events" className="text-accent-600 font-bold">Back to Events</Link>
              </div>
            );
          }

          return (
            <div className="max-w-md mx-auto py-16 px-4">
              <h1 className="text-2xl font-bold mb-2">{event.title}</h1>
              <p className="text-gray-500 mb-6">{event.location} • {new
Date(event.date).toLocaleDateString()}</p>

              {error && <p className="bg-red-50 text-red-700 p-3 rounded-lg mb-4 text-sm">{error}</p>}

              <form onSubmit={handleSubmit} className="space-y-4">
                <input
                  name="name" required placeholder="Full Name" value={formData.name}
                  onChange={handleChange} className="w-full border rounded-lg p-2.5"
                />
                <input
                  name="email" type="email" required placeholder="Email" value={formData.email}
                  onChange={handleChange} className="w-full border rounded-lg p-2.5"
                />
                <input
                  name="phone" required placeholder="Phone Number" value={formData.phone}
                  onChange={handleChange} className="w-full border rounded-lg p-2.5"
                />
                <select
                  name="experienceLevel" required value={formData.experienceLevel}
```

```
                onChange={handleChange} className="w-full border rounded-lg p-2.5"
              >
                <option value="">Select Experience Level</option>
                <option value="Beginner">Beginner</option>
                <option value="Intermediate">Intermediate</option>
                <option value="Advanced">Advanced</option>
              </select>

              <button
                type="submit" disabled={submitting}
                className="w-full bg-primary-900 text-white py-3 rounded-lg font-bold
disabled:opacity-50"
              >
                {submitting ? 'Submitting...' : 'Submit Registration'}
              </button>
            </form>
          </div>
        );
      };

      export default CourseRegistration;
```

This one is a **controlled form** — every input's `value` comes from the `formData` state object, and `handleChange` updates just that one field using the input's `name` attribute and a spread (`{ ...formData, [e.target.name]: e.target.value }`). This is the pattern to learn well, since almost every form you'll ever build in React follows it.

frontend/src/pages/Contact.jsx

```
      import React, { useState } from 'react';

      const Contact = () => {
        const [sent, setSent] = useState(false);

        const handleSubmit = (e) => {
          e.preventDefault();
          // NOTE: this doesn't actually send anything anywhere yet — there's no
          // backend route for it. Wiring it up for real just needs one more
          // Express route and swapping this for a real api.post() call.
          setSent(true);
        };

        if (sent) {
          return <p className="text-center py-20 font-bold">Thanks! We'll get back to you soon.</p>;
        }

        return (
          <div className="max-w-md mx-auto py-16 px-4">
            <h1 className="text-2xl font-bold mb-6">Contact Us</h1>
            <form onSubmit={handleSubmit} className="space-y-4">
              <input required placeholder="Your Name" className="w-full border rounded-lg p-2.5" />
              <input required type="email" placeholder="Your Email" className="w-full border rounded-
lg p-2.5" />
              <textarea required rows="4" placeholder="Message" className="w-full border rounded-lg p-
2.5"></textarea>
              <button className="w-full bg-primary-900 text-white py-3 rounded-lg font-bold">Send
Message</button>
            </form>
          </div>
        );
      };

      export default Contact;
```

frontend/src/pages/AdminLogin.jsx

```
      import React, { useState, useEffect } from 'react';
```

```
        import { useNavigate } from 'react-router-dom';
        import { useAuth } from '../context/AuthContext';

        const AdminLogin = () => {
          const [email, setEmail] = useState('');
          const [password, setPassword] = useState('');
          const [error, setError] = useState(null);
          const [loading, setLoading] = useState(false);
          const { admin, login } = useAuth();
          const navigate = useNavigate();

          useEffect(() => {
            if (admin) navigate('/admin', { replace: true });
          }, [admin, navigate]);

          const handleSubmit = async (e) => {
            e.preventDefault();
            setError(null);
            setLoading(true);
            try {
              await login(email, password);
              navigate('/admin');
            } catch (err) {
              setError(err.message);
            } finally {
              setLoading(false);
            }
          };

          return (
            <div className="max-w-sm mx-auto py-24 px-4">
              <h1 className="text-2xl font-bold mb-6 text-center">Admin Login</h1>
              {error && <p className="bg-red-50 text-red-700 p-3 rounded-lg mb-4 text-sm">{error}</p>}
              <form onSubmit={handleSubmit} className="space-y-4">
                <input
                  type="email" required placeholder="Email" value={email}
                  onChange={(e) => setEmail(e.target.value)} className="w-full border rounded-lg p-2.5"
                />
                <input
                  type="password" required placeholder="Password" value={password}
                  onChange={(e) => setPassword(e.target.value)} className="w-full border rounded-lg p-
2.5"
                />
                <button disabled={loading} className="w-full bg-primary-900 text-white py-3 rounded-lg
font-bold disabled:opacity-50">
                  {loading ? 'Logging in...' : 'Log In'}
                </button>
              </form>
            </div>
          );
        };

        export default AdminLogin;
```

---

17. Frontend, Step 8: The Admin Dashboard

The original production version crammed five tabs into one 800-line file. Here we do the same five tabs (Events, Registrations, Secretaries, Alumni, Gallery), but since every tab shares the exact same “form + table” shape, and we removed file uploads and CSV export, the whole thing is much shorter — you’ll be able to actually read the whole file in one sitting.

Step 1 — State and data fetching

```
        import React, { useState, useEffect } from 'react';
```

```
import { api } from '../utils/api';
import { useAuth } from '../context/AuthContext';

const AdminDashboard = () => {
  const { admin, logout } = useAuth();
  const [activeTab, setActiveTab] = useState('events');

  const [events, setEvents] = useState([]);
  const [secretaries, setSecretaries] = useState([]);
  const [alumni, setAlumni] = useState([]);
  const [gallery, setGallery] = useState([]);
  const [registrations, setRegistrations] = useState([]);
  const [selectedEventId, setSelectedEventId] = useState('');

  const [editItem, setEditItem] = useState(null);   // item currently being edited (or null)
  const [showForm, setShowForm] = useState(false);  // is the "add new" form open?
  const [formData, setFormData] = useState({});     // the form's current values

  const fetchData = async () => {
    if (activeTab === 'events') setEvents(await api.get('/events'));
    if (activeTab === 'secretaries') setSecretaries(await api.get('/secretaries'));
    if (activeTab === 'alumni') setAlumni(await api.get('/alumni'));
    if (activeTab === 'gallery') setGallery(await api.get('/gallery'));
    if (activeTab === 'registrations') {
      const evts = await api.get('/events');
      setEvents(evts);
      const id = selectedEventId || evts[0]?._id;
      if (id) {
        setSelectedEventId(id);
        setRegistrations(await api.get(`/registrations/event/${id}`));
      }
    }
  };

  useEffect(() => { fetchData(); }, [activeTab]);

  useEffect(() => {
    if (activeTab === 'registrations' && selectedEventId) {
      api.get(`/registrations/event/${selectedEventId}`).then(setRegistrations);
    }
  }, [selectedEventId]);
```

Five lists, one per tab — all starting empty and filled in by `fetchData()`, which is re-run automatically whenever `activeTab` changes.

Step 2 — Generic create / edit / delete handlers

```
  const openAddForm = () => {
    setFormData({});
    setEditItem(null);
    setShowForm(true);
  };

  const openEditForm = (item) => {
    setFormData(item);
    setEditItem(item);
    setShowForm(true);
  };

  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e, endpoint) => {
    e.preventDefault();
    if (editItem) {
      await api.put(`${endpoint}/${editItem._id}`, formData);
    } else {
      await api.post(endpoint, formData);
    }
```

```
    setShowForm(false);
    setEditItem(null);
    fetchData();
  };

  const handleDelete = async (endpoint, id) => {
    if (!window.confirm('Delete this item?')) return;
    await api.delete(`${endpoint}/${id}`);
    fetchData();
  };
```

This is the key beginner-friendly idea: **one** `handleSubmit` and **one** `handleDelete` work for every single tab, because they just take the API endpoint as an argument. `editItem` being `null` or holding a real object is what decides whether we `POST` (create) or `PUT` (update) — and `formData` is a normal **controlled** state object shared by whichever form is currently open, updated field-by-field through `handleChange`.

Step 3 — The tab bar and the Events tab (shown in full)

```
  return (
    <div className="max-w-5xl mx-auto py-10 px-4">
      <div className="flex justify-between items-center mb-6">
        <div>
          <h1 className="text-xl font-bold">Admin Dashboard</h1>
          <p className="text-xs text-gray-500">Logged in as {admin?.email}</p>
        </div>
        <button onClick={logout} className="text-sm font-bold text-red-600">Log Out</button>
      </div>

      <div className="flex gap-2 mb-6 flex-wrap">
        {['events', 'registrations', 'secretaries', 'alumni', 'gallery'].map((tab) => (
          <button
            key={tab}
            onClick={() => { setActiveTab(tab); setShowForm(false); setEditItem(null); }}
            className={`px-4 py-2 rounded-lg text-sm font-semibold capitalize ${
              activeTab === tab ? 'bg-primary-900 text-white' : 'bg-white border'
            }`}
          >
            {tab}
          </button>
        ))}
      </div>

      {activeTab === 'events' && (
        <div>
          <button onClick={openAddForm} className="mb-4 bg-green-700 text-white px-4 py-2
rounded-lg text-sm font-bold">
            + Add Event
          </button>

          {showForm && (
            <form onSubmit={(e) => handleSubmit(e, '/events')} className="bg-white border
rounded-xl p-4 mb-6 space-y-3">
              <input name="title" required placeholder="Title" value={formData.title || ''}
onChange={handleChange} className="w-full border rounded-lg p-2" />
              <select name="type" required value={formData.type || ''} onChange={handleChange}
className="w-full border rounded-lg p-2">
                <option value="">Select Type</option>
                <option value="course">Course</option>
                <option value="expedition">Expedition</option>
                <option value="competition">Competition</option>
                <option value="other">Other</option>
              </select>
              <input name="date" type="date" required value={formData.date ?
formData.date.slice(0, 10) : ''} onChange={handleChange} className="w-full border rounded-lg p-
2" />
              <input name="location" required placeholder="Location" value={formData.location ||
''} onChange={handleChange} className="w-full border rounded-lg p-2" />
              <textarea name="description" required placeholder="Description" value=
{formData.description || ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
```

```
                    <input name="capacity" type="number" placeholder="Capacity (optional)" value=
{formData.capacity || ''} onChange={handleChange} className="w-full border rounded-lg p-2" />

                    <div className="flex gap-2">
                      <button type="button" onClick={() => setShowForm(false)} className="border px-4
py-2 rounded-lg text-sm">Cancel</button>
                      <button type="submit" className="bg-primary-900 text-white px-4 py-2 rounded-lg
text-sm font-bold">Save</button>
                    </div>
                  </form>
                )}

                <div className="space-y-2">
                  {events.map((event) => (
                    <div key={event._id} className="bg-white border rounded-lg p-3 flex justify-
between items-center">
                      <div>
                        <p className="font-bold text-sm">{event.title}</p>
                        <p className="text-xs text-gray-500">{event.registrationsCount} registered</p>
                      </div>
                      <div className="flex gap-3 text-sm">
                        <button onClick={() => openEditForm(event)} className="text-primary-600 font-
bold">Edit</button>
                        <button onClick={() => handleDelete('/events', event._id)} className="text-
red-600 font-bold">Delete</button>
                      </div>
                    </div>
                  ))}
                </div>
              </div>
            )}
```

Every input uses `value={formData.title || ''}` and `onChange={handleChange}` — a normal controlled input, kept in sync with the shared `formData` object. Opening “Edit” on a row just calls `openEditForm(event)`, which drops that event straight into `formData` — so the exact same form re-appears, pre-filled.

Step 4 — Registrations tab (read-only, plus a payment toggle)

```
            {activeTab === 'registrations' && (
              <div>
                <select
                  value={selectedEventId}
                  onChange={(e) => setSelectedEventId(e.target.value)}
                  className="border rounded-lg p-2 mb-4 bg-white"
                >
                  {events.map((e) => <option key={e._id} value={e._id}>{e.title}</option>)}
                </select>

                <div className="space-y-2">
                  {registrations.map((reg) => (
                    <div key={reg._id} className="bg-white border rounded-lg p-3 flex justify-between
items-center">
                      <div>
                        <p className="font-bold text-sm">{reg.name}</p>
                        <p className="text-xs text-gray-500">{reg.email} • {reg.phone}</p>
                      </div>
                      <div className="flex items-center gap-3">
                        <button
                          onClick={async () => {
                            const next = reg.paymentStatus === 'Pending' ? 'Received' : 'Pending';
                            await api.put(`/registrations/${reg._id}/payment`, { paymentStatus: next
});

                            fetchData();
                          }}
                          className={`text-xs font-bold px-2 py-1 rounded ${
                            reg.paymentStatus === 'Received' ? 'bg-green-100 text-green-700' : 'bg-
orange-100 text-orange-700'
                          }`}
                        >
```

```
                          {reg.paymentStatus}
                        </button>
                        <button onClick={() => handleDelete('/registrations', reg._id)}
className="text-red-600 text-sm font-bold">Delete</button>
                      </div>
                    </div>
                  ))}
                </div>
              </div>
            )}
```

No add-form here — registrations only ever come in through the public registration page, never typed in by an admin. The payment badge doubles as a button that flips between “Pending” and “Received”.

Step 5 — Secretaries, Alumni, and Gallery tabs

These three repeat the exact same “Add button → form → list with Edit/Delete” shape from the Events tab — only the field names change:

```
            {activeTab === 'secretaries' && (
              <div>
                <button onClick={openAddForm} className="mb-4 bg-green-700 text-white px-4 py-2
rounded-lg text-sm font-bold">+ Add Entry</button>
                {showForm && (
                  <form onSubmit={(e) => handleSubmit(e, '/secretaries')} className="bg-white border
rounded-xl p-4 mb-6 space-y-3">
                    <input name="yearStart" type="number" required placeholder="Start Year" value=
{formData.yearStart || ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                    <input name="yearEnd" type="number" required placeholder="End Year" value=
{formData.yearEnd || ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                    <input name="name" required placeholder="Secretary Name" value={formData.name ||
''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                    <input name="note" placeholder="Note (optional)" value={formData.note || ''}
onChange={handleChange} className="w-full border rounded-lg p-2" />
                    <div className="flex gap-2">
                      <button type="button" onClick={() => setShowForm(false)} className="border px-4
py-2 rounded-lg text-sm">Cancel</button>
                      <button type="submit" className="bg-primary-900 text-white px-4 py-2 rounded-lg
text-sm font-bold">Save</button>
                    </div>
                  </form>
                )}
                <div className="space-y-2">
                  {secretaries.map((sec) => (
                    <div key={sec._id} className="bg-white border rounded-lg p-3 flex justify-between
items-center">
                      <p className="text-sm">{sec.yearStart}–{sec.yearEnd}: <strong>{sec.name}
</strong></p>
                      <div className="flex gap-3 text-sm">
                        <button onClick={() => openEditForm(sec)} className="text-primary-600 font-
bold">Edit</button>
                        <button onClick={() => handleDelete('/secretaries', sec._id)} className="text-
red-600 font-bold">Delete</button>
                      </div>
                    </div>
                  ))}
                </div>
              </div>
            )}

            {activeTab === 'alumni' && (
              <div>
                <button onClick={openAddForm} className="mb-4 bg-green-700 text-white px-4 py-2
rounded-lg text-sm font-bold">+ Add Alumni</button>
                {showForm && (
                  <form onSubmit={(e) => handleSubmit(e, '/alumni')} className="bg-white border
rounded-xl p-4 mb-6 space-y-3">
                    <input name="name" required placeholder="Name" value={formData.name || ''}
onChange={handleChange} className="w-full border rounded-lg p-2" />
                    <input name="batchYear" type="number" required placeholder="Batch Year" value=
```

```
{formData.batchYear || ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                      <input name="roleHistory" required placeholder="Role History" value=
{formData.roleHistory || ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                      <input name="photoUrl" placeholder="Photo URL (paste a link)" value=
{formData.photoUrl || ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                      <div className="flex gap-2">
                        <button type="button" onClick={() => setShowForm(false)} className="border px-4
py-2 rounded-lg text-sm">Cancel</button>
                        <button type="submit" className="bg-primary-900 text-white px-4 py-2 rounded-lg
text-sm font-bold">Save</button>
                      </div>
                    </form>
                  )}
                  <div className="space-y-2">
                    {alumni.map((a) => (
                      <div key={a._id} className="bg-white border rounded-lg p-3 flex justify-between
items-center">
                        <p className="text-sm font-bold">{a.name} <span className="text-gray-400 font-
normal">({a.batchYear})</span></p>
                        <div className="flex gap-3 text-sm">
                          <button onClick={() => openEditForm(a)} className="text-primary-600 font-
bold">Edit</button>
                          <button onClick={() => handleDelete('/alumni', a._id)} className="text-red-600
font-bold">Delete</button>
                        </div>
                      </div>
                    ))}
                  </div>
                </div>
              )}

              {activeTab === 'gallery' && (
                <div>
                  <button onClick={openAddForm} className="mb-4 bg-green-700 text-white px-4 py-2
rounded-lg text-sm font-bold">+ Add Photo</button>
                  {showForm && (
                    <form onSubmit={(e) => handleSubmit(e, '/gallery')} className="bg-white border
rounded-xl p-4 mb-6 space-y-3">
                      <input name="imageUrl" required placeholder="Image URL" value={formData.imageUrl
|| ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                      <input name="year" type="number" required placeholder="Year" value={formData.year
|| ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                      <input name="eventTag" required placeholder="Tag (e.g. Rock Climbing)" value=
{formData.eventTag || ''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                      <input name="caption" placeholder="Caption (optional)" value={formData.caption ||
''} onChange={handleChange} className="w-full border rounded-lg p-2" />
                      <div className="flex gap-2">
                        <button type="button" onClick={() => setShowForm(false)} className="border px-4
py-2 rounded-lg text-sm">Cancel</button>
                        <button type="submit" className="bg-primary-900 text-white px-4 py-2 rounded-lg
text-sm font-bold">Save</button>
                      </div>
                    </form>
                  )}
                  <div className="grid grid-cols-3 sm:grid-cols-4 gap-3">
                    {gallery.map((item) => (
                      <div key={item._id} className="relative border rounded-lg overflow-hidden">
                        <img src={item.imageUrl} className="w-full h-24 object-cover" />
                        <button
                          onClick={() => handleDelete('/gallery', item._id)}
                          className="absolute top-1 right-1 bg-white/90 text-red-600 text-xs px-1.5
rounded"
                        >
                          ×
                        </button>
                      </div>
                    ))}
                  </div>
                </div>
              )}
            </div>
          );
```

```
};

export default AdminDashboard;
```

Gallery items only get a delete button, not an edit form — same reasoning as the original: it's simple enough that "delete and re-add" is good enough for a photo grid.

---

18. Running Everything Together

1. Make sure MongoDB is installed and running (or use a free MongoDB Atlas cloud database).
2. Create `backend/.env` with `PORT`, `MONGO_URI`, and `JWT_SECRET`.
3. From the project root: `npm run install-all`.
4. Run `npm run seed` to fill the database with starter data.
5. Run `npm run dev` — this starts Express on port 5000 and Vite on port 5173 at the same time.
6. Open `http://localhost:5173` for the public site, or go to `/admin/login` and sign in with `admin@jumhc.org` / `admin1966`.

Tracing one request through the whole stack

Here's exactly what happens when someone opens the Events page, tying every section together:

1. React Router matches `/events` to `<Events />` (Section 15).
2. `Events.jsx`'s `useEffect` calls `api.get('/events')` (Section 16).
3. `utils/api.js` (Section 12) sends a real `fetch()` to `http://localhost:5000/api/events`.
4. Express (`server.js`, Section 5) routes that to `routes/events.js` (Section 8), which runs `Event.find()`.
5. Mongoose (using the schema from Section 6) turns that into a real MongoDB query and returns the matching documents.
6. The route sends them back as JSON with `res.json(events)`.
7. Back in the browser, `api.get()` resolves with that data, `Events.jsx` calls `setEvents(data)`, and React re-renders the page with real events on screen.

Every single page in this project, and every button in the admin dashboard, is some variation of this exact round trip. Once you can trace it confidently, you understand the MERN stack.

---

19. What to Add Next, Once This Feels Easy

This version was kept intentionally simple so you can build it end-to-end and actually understand every line. Once it works and feels comfortable, here are the "real production" features from the fuller version worth adding one at a time, each as its own small learning project:

1. **File uploads with Multer** — let admins upload a real photo instead of pasting a URL.
2. **Sending real emails with Nodemailer** — actually email a registration confirmation.
3. **Atomic seat-counting** — fix the tiny race condition in the registrations route using MongoDB's `$expr` + `$inc` in one atomic update.
4. **CSV export** — let admins download a spreadsheet of registrations.
5. **Role-based permissions** — add an "Editor" role that can't delete things, alongside a "Super Admin" who can.
6. **Search & sort everywhere** — add the extra filters (year, active-instructor-only, mobile menu) we trimmed out here.

Adding each of these one at a time, to a project you already fully understand, is a much better way to learn them than trying to absorb all of it at once.